Pairwise Distances Under a Learned Metric — Write-up

Zihe (Leo) Liu · November 2026 · repo: github.com/ZiheLiu0424/Einsummable

All numbers are from `harness/results.csv`, produced by `scripts/run_grid.sh` on CLEAR node **agate** (Xeon E5-2640 v4, AVX2 + FMA, no AVX-512), pinned to CPU 0; GFLOP/s is the harness's `useful_flops()/t_min`. Every kernel except k1 fills only the upper triangle (j ≥ i). Every rung passes both checks wherever it runs (k0 is skipped at 5 points); k1 is the harness's unused template. Figure: `harness/ladder.png`.

(a) Algorithm and cost model

Naive: for each of the n(n+1)/2 pairs form $x_i - x_j$ (d flops), multiply by M (2dk) and take the norm (2k): **$\approx n^2dk$**. The waste is that $x_i M$ is recomputed n times, because $(x_i - x_j)M = y_i - y_j$.

- **A: Y = XM, then Gram.** $D_{ij} = r_i + r_j - 2\, y_i \cdot y_j$ with $r_i = \|y_i\|^2$. Cost 2ndk (XM) + n(n+1)k (upper-triangle Gram) + $O(n^2)$ epilogue.
- **B: A = MM^s, then XAX^s.** Cost $2d^2k + 2nd^2 + n(n+1)d$.

The dominant terms are n^2k (A) and n^2d (B), so A wins whenever k ≤ d, i.e. on the whole grid: by 16× at k = 32 and by 1–25% at k = 512 (where C_B = C_A + $2d^3$). A is 250–460× cheaper than naive. Within A, Gram/XM ≈ n/(2d), so at n = 1024 the XM product is half the work and has to be fast too.

Prediction check: the harness refuses $n^2dk > 4\times10^{11}$, so k0 should run only at (1024, all k) and (4096, 32). Measured: exactly that, with time 3.97 → 16.9 → 59.8 s for k = 32 → 128 → 512 and ×16.1 from n = 1024 to 4096.

Numerics: A and B both subtract large, nearly equal numbers. For the planted near-duplicates (true D ≈ 10^{-13}) the rounding error $\varepsilon(r_i + r_j)$ exceeds the answer: with the clamp removed, k2 returns 1–22 negative sampled entries, still PASS because the tolerance is relative to $\|y_i\|^2 + \|y_j\|^2$. All kernels clamp with max(0, ·) and write the diagonal as 0, so `neg` = 0. The clamp makes the value legal, not more accurate.

(b) The ladder

rung	idea	GFLOP/s on the grid
k0	naive, scalar	0.01–0.02
k2	algorithm A, plain loops, epilogue fused into the Gram loop	2.1–5.4
k3	AVX2 4×8 register tile for both XM and Gram; Y^T copy	6.9–24.1
k4	+ MC/KC/NC cache blocking, packed A/B panels	14.1–34.0
k5	+ row-order tile sweep when kc ≤ 64	20.4–33.5
blas	OpenBLAS dgemm + dsyrk + separate epilogue pass	13.4–36.6

k2. Expected hundreds× over k0; measured 210–360× in time. Still only 2–5 GFLOP/s: each Gram entry is a k-long reduction into one scalar, so the compiler's SIMD helps little.

k3. The micro-kernel holds a 4×8 C tile in 8 YMM accumulators; each K step does 2 loads of B, 4 broadcasts of A and 8 FMAs (11 of 16 registers). 3–6× over k2. It collapses to 6.9 at n = 8192, k = 512: the 8-column strip of Y^T that one tile walks spans 512 rows 64 KB apart, 32 MB in total, more than the 25 MB L3, so B streams from DRAM for every row block. At n = 4096 Y^T is 16 MB, fits in L3, and k3 holds 14.7.

k4. GotoBLAS-style blocking and packing with KC = 256, MC = 96, NC = 2048: a B micro-panel (KC×8 = 16 KiB) stays in L1 and the A block in L2. It nearly doubled k3 at k ≥ 128 (n = 8192, k = 512: 6.9 → 29.0). Unexpectedly it was *no better or slightly worse* at k = 32 (n = 4096: 14.8 → 14.1). Explanation: the BLIS jr-outer/ir-inner order writes C in 4×8 tiles going *down* a column strip; at small k each tile has little compute (2·32·32 flops) per 256 B written, so the scattered writes dominate.

k5. When kc ≤ 64, sweep tiles along rows so stores are sequential; otherwise identical to k4. At k = 32: 14.1 → 20.4 (n = 4096) and 15.6 → 20.9 (n = 8192); unchanged at k ≥ 128, as expected. At k = 32 k5

beats OpenBLAS at every n (25.1/20.4/20.9 vs 20.6/15.3/13.4): OpenBLAS writes G and then makes a second pass over n^2 doubles for the epilogue, which we fuse away.

(c) Where we are and why

Theoretical peak: 2 FMA ports × 4 doubles × 2 flops = 16 flops/cycle, i.e. 38.4 GFLOP/s at the 2.4 GHz base and 54.4 at the 3.4 GHz max turbo. Measured dgemm ceiling: 39.2–40.0 GFLOP/s, slightly below the 41–43 quoted for an idle node; it corresponds to ~2.5 GHz sustained, so the core runs AVX-heavy code well below max turbo. The gap to 54.4 is mostly clock, not OpenBLAS.

Best kernel (k5): 70–76% of the dgemm ceiling at k = 512, 59–84% at k = 128, 51–63% at k = 32.

- **k = 512: compute-bound by the micro-kernel shape.** FMA latency on Broadwell is 5 cycles with 2 ports, so 10 independent accumulators are needed to keep both busy. The 4×8 tile has 8, capping it near 8/10 of peak ≈ 32 GFLOP/s at the ceiling's clock. We measure 28–30; OpenBLAS's larger tile reaches 35–36.6. (n = 1024, k = 128 hits 34, so the cap is approximate.)
- **k = 128 and 32: the output write matters.** The upper triangle of D is 4 MB at n = 1024 (fits in L3) but 64–256 MB at n ≥ 4096. Both k5 and OpenBLAS drop ~25% from n = 1024 to n ≥ 4096 at k = 128 (34 → 24–25, 35 → 28); at k = 32 the drop is 25 → 20. At k = 32 each entry carries 64 flops and ~16 B of DRAM traffic (store + write-allocate read), i.e. 4 flops/B. At n = 8192 k5 takes 0.116 s; the micro-kernel alone at ~32 GFLOP/s would need ~0.075 s, and the remaining ~0.04 s is consistent with writing ~0.54 GB. I have not measured the node's single-core store bandwidth, so this is an argument, not a proof.
- Caveat: the n = 8192, k ∈ {32, 128} rows ran at node load 8–12 (another user); t_min and t_med still agree within 1–8%, so core 0 was not heavily contended.

(d) With eight more hours (in order of expected payoff)

1. **A 12-accumulator micro-kernel (6×8 or 4×12).** Hides FMA latency; by the argument in (c) it should move k ≥ 128 from ~80% toward ~90% of the ceiling.
2. **Non-temporal stores (`_mm256_stream_pd`) for D at small k.** Removes the write-allocate read, up to half the DRAM traffic at k = 32. A STREAM/memset measurement first would confirm the bandwidth bound.
3. **Tune MC/KC/NC per k.** In particular KC = k at k = 512 avoids the second pass over partial sums held in D.
4. **Re-run n = 8192 on an idle node**, with an idle hyperthread sibling.

(e) How I used AI

I used Claude (in a Claude Code project) throughout. What I asked for: a summary of the spec and the Step 0 cost model; kernels k2–k5, each as a single-idea rung; SSH-key setup for CLEAR and running the grid from my laptop; a first draft of this write-up; and a step-by-step animated lesson that puts the spec's own sentences and the real code lines next to every idea. What I kept: all four rungs (k2–k5) as written, after reading each file with the lesson, and the cost model, which I checked against k0's measured times. What I threw out: the first version of the lesson, which I could not follow; it was redone twice, slower and tied to the code. To be clear, the AI wrote the code; my work was understanding it well enough to explain every line and every number here, and this conversation is the test of that.

(f) Time log

when	what	hours
Nov 2	read the spec and harness README, set up the repo and the plan	1.0
Nov 5	Step 0 cost model; read k2–k5 as they were written and tested	1.5
Nov 5	CLEAR SSH setup, build, full grid run (9 points)	2.0
Nov 5–12	write-up (draft and final); worked through the lesson (three versions) and its exercises; interview prep	3.5
	total	**8.0**

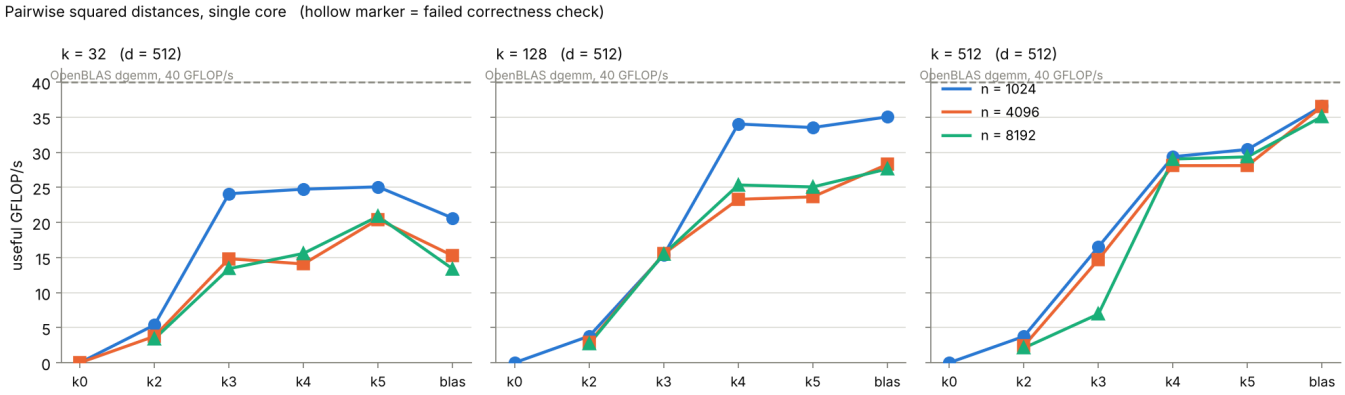


Figure: useful GFLOP/s per kernel, one panel per k, one line per n; dashed line = measured dgemm ceiling (harness/ladder.png).